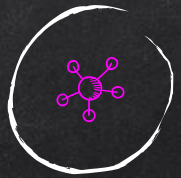
SÉCURITÉ



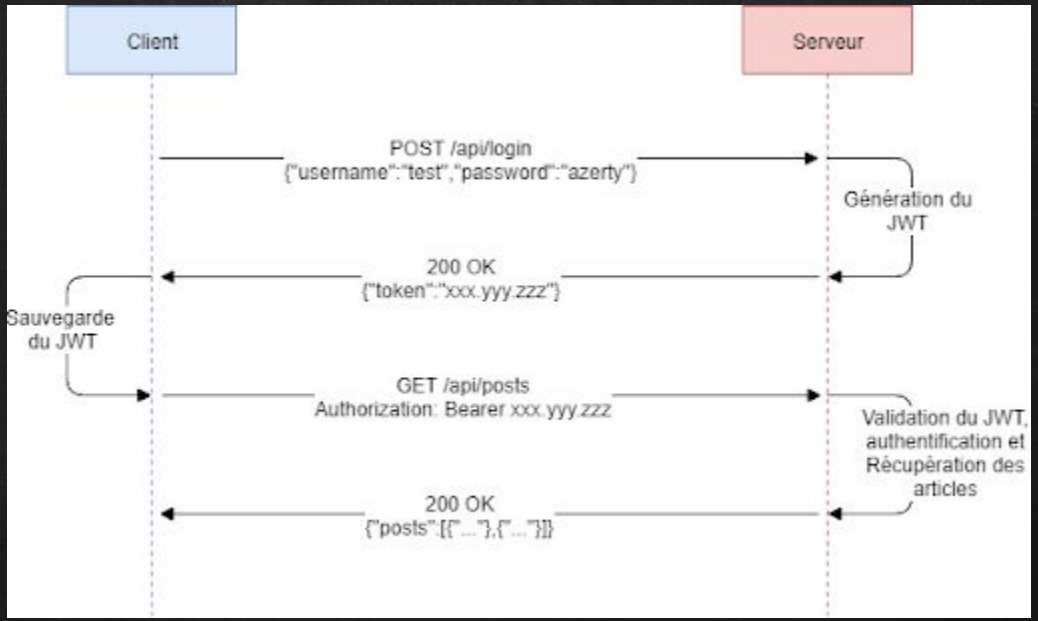
API REST
Sécurité

AUTHENTIFICATION

Qui est l'utilisateur ?

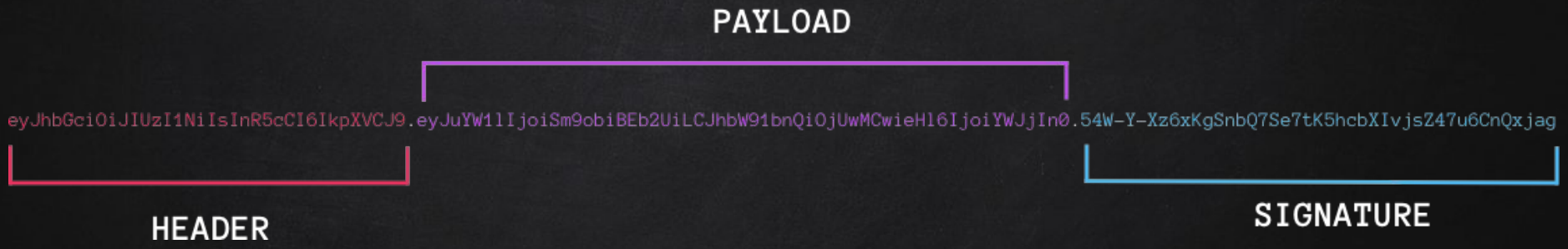
AUTHORIZATION

Quelles sont les permissions de l'utilisateur ?



Source: https://eric.munier.me/jwt/

JWT TOKEN



Source: codeburst.io

Source:
https://caketuts.key-conseil.fr/index.php/2017/01/29/implementer-lauthentification-par-jwt/

Génération du Token JWT

```
private string GenerateJwtToken(string username, List<string> roles) {
        //Add User Infos
        var claims = new List<Claim>(){
            new Claim(JwtRegisteredClaimNames.Sub, username),
            new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
            new Claim(ClaimTypes.NameIdentifier, username)
        };
        //Add Roles
        roles.ForEach(role => {
            claims.Add(new Claim(ClaimTypes.Role, role));
        });
        //Signin Key
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration["JwtKey"]));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
        //Expiration time
        var expires = DateTime.Now.AddDays(Convert.ToDouble(_configuration["JwtExpireDays"]));
        //Create JWT Token Object
        var token = new JwtSecurityToken(
            _configuration["JwtIssuer"],
            _configuration["JwtIssuer"],
            claims,
            expires: expires,
            signingCredentials: creds
        );
        //Serializes a JwtSecurityToken into a JWT in Compact Serialization Format.
        return new JwtSecurityTokenHandler().WriteToken(token);
    }
```

Ajout du service

```
// Méthode de la classe Startup pour ajouter nos services dans l'injecteur de dépendance
public void ConfigureServices(IServiceCollection services) {
    ...
  //JWT Authentication
   services
   .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
   .AddJwtBearer(options => {
       options.TokenValidationParameters = new TokenValidationParameters()
       {
           ValidateIssuer = false,
           ValidateAudience = false,
           ValidAudience = Configuration["JwtIssuer"],
           ValidIssuer = Configuration["JwtIssuer"],
           ValidateIssuerSigningKey = true,
           IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(Configuration["JwtKey"])),
           //Retourne la différence de temps maximale autorisée entre le client et les paramètres de l'horloge du serveur.
           ClockSkew = TimeSpan.Zero // remove delay of token when expire
       };
   });
    ...
}
```

AJOUT DU MIDDLEWARE

```
// This method gets called by the runtime. Use this method to configure the HTTP request pipeline.
public void Configure(IApplicationBuilder app, IWebHostEnvironment env){
    ...
    //Add Authentofication => code Erreur 401
    app.UseAuthentication();

    //Authorization => Code Erreur 403 Forbidden
    app.UseAuthorization();
    ...
}
```

Fichier de configuration

appsettings.json

```
{
  ...

  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=DatabaseName;Trusted_Connection=True;"
  },

  "JwtKey": "mySecretPassword",
  "JwtIssuer": "https://localhost:5001", //replace this with your application url
  "JwtExpireDays": 30
}
```

Sécuriser route et controleur

Liste des différentes stratégie de sécurité :

- ✘ Autorisation basée sur les rôles
- ✘ Autorisation basée sur les revendications
- ✘ Autorisation basée sur la stratégie